

Less Budget, More Memory: Diagnosing and Mitigating Non-Monotonic Activation Memory in PyTorch’s Memory-Budget Partitioner

Siddharth Ajith
Independent Researcher
siddharthajith07@gmail.com

October 2026

Abstract

`torch.compile` exposes an `activation_memory_budget` knob that trades recomputation for memory during training. We show that the peak memory it produces is not monotone in the knob: on a 32-layer Llama-style model compiled with Inductor, a budget of 0.05 uses $2.5\times$ the peak memory of a budget of 0.13 (904 vs. 363 MB), and a BERT model shows the same non-monotonicity below budget 0.05. We trace this to two limitations of AOTAutograd’s min-cut partitioning pipeline. In the first (*selection*), the FLOP-maximizing knapsack that decides what to save spends small budgets on attention outputs, leaving the residual stream to be recomputed. In the second (*lifetime*), each recomputed value is defined once in the emitted backward graph, at its first backward use, and held until its last. Inductor’s own memory estimator ranks all 14 plans we test in the same order as the measured peak (0.7% median error); the partitioner’s in-tree evaluator does not. We then evaluate a post-partition, use-site rematerialization pass in the style of XLA’s, which recomputes cheap values again near their late uses and leaves the saved set unchanged. At the same budget it reduces Llama’s Inductor peak by 17–28% at no measurable step-time cost, removing about half of the excess; a variant that also duplicates matrix multiplies goes 7.5–20% below the best stock budget on a dense grid of budgets, for 6–18% more step time, and makes the budget curve monotone over the budgets we tested (0.05–0.30). On BERT the excess is held dropout masks, which the pass does not regenerate because it never duplicates random operations, so its effect there is small (0–3%). Gradients of the rematerialization pass are checked against eager execution (Llama, every budget) or an unbudgeted run (BERT). We also reproduce a known bug in which recomputed dropout silently corrupts gradients under a memory budget, measure the memory cost of its proposed fix, and find that, with that fix applied, the partitioner can trigger a second, CUDA-specific random-number bug in Inductor (PyTorch issue #198333). All results come from one consumer GPU, two small models and fp32.

1 Introduction

Activation checkpointing trades compute for memory: instead of storing every intermediate activation for the backward pass, some are recomputed from saved values [4]. PyTorch 2’s compiler stack [1] automates this choice. AOTAutograd traces a joint forward–backward graph, and its min-cut partitioner decides which values the forward pass saves. A single scalar, `torch._functorch.config.activation_memory_budget` $\in [0, 1]$, lets users interpolate between the default plan (budget 1) and maximal recomputation (budget 0) [24]. Internally the partitioner solves a 0/1 knapsack: candidate values are items, their size is the weight and their estimated recomputation runtime is the value [12, 19]. The budget is defined in terms of saved-activation bytes, not peak memory, so nothing promises that peak memory falls with it; the partitioner’s design note states that it ignores memory usage within each graph [5]. Still, a user lowering the budget expects it to.

Figure 1 shows that it need not: for a 32-layer Llama-style model the measured peak *rises* from 363 MB at budget 0.13, the lowest point of a dense grid of budgets, to 904 MB at budget 0.05 under Inductor (dashed gray; the hollow markers are the dense grid). The author reported this as PyTorch issue #197838 [22], and another user has since reproduced a non-monotone curve on a GPT-style model on different hardware [11]. This report explains why it happens and evaluates ways to mitigate it.

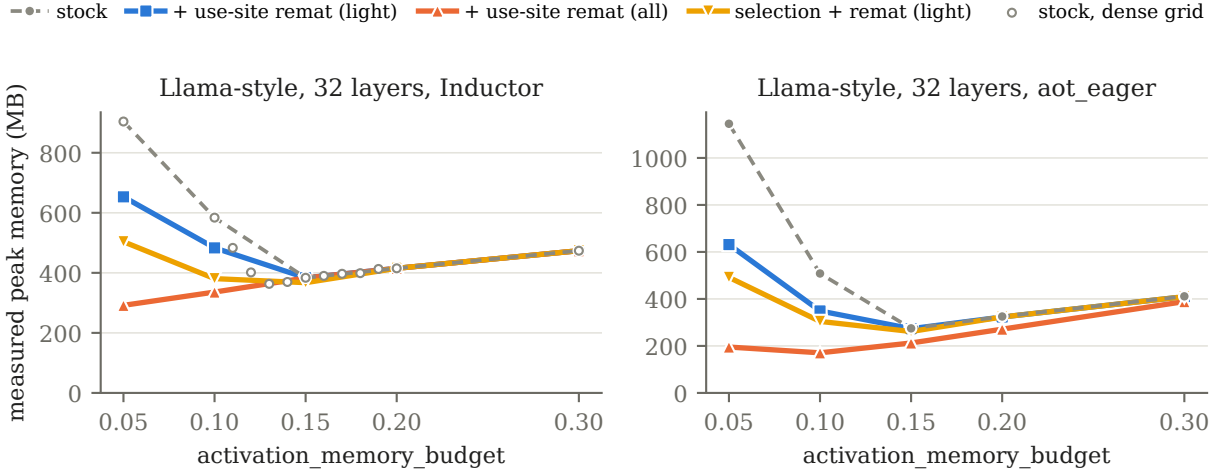


Figure 1: Measured peak allocated memory of one training step versus `activation_memory_budget`. Stock PyTorch (dashed; hollow markers: a denser stock-only grid) is non-monotone: lowering the budget below 0.13 *increases* peak memory. Light use-site rematerialization (Section 4.1) removes about half of the excess under Inductor without changing what is saved; its all-ops variant makes the Inductor curve monotone over the tested budgets (0.05–0.30). Where the curves coincide, the pass changes nothing. The `aot_eager` panel is a partitioner-only ablation (Section 3.1). Numbers in Table 2.

Contributions.

1. **Diagnosis** (Section 3). We decompose the anomaly into a selection limitation and a lifetime limitation, with interventions under Inductor: forcing MLP outputs into the saved set lowers the low-budget peak monotonically (904→671 MB), and splitting recomputed lifetimes with the saved set held fixed removes about half of the excess. A sweep of sequence length and MLP size shows the selection limitation appearing only where the knapsack values attention outputs above MLP outputs, and the lifetime limitation persisting below that point.
2. **Peak prediction** (Section 3.3). Inductor’s `memory.py` estimator ranks 14 plans exactly (Spearman 1.000, 0.7% median error). A static liveness walk over the partitioner’s emitted graphs is accurate (0.4%) when the backend executes them as emitted, but under Inductor it only nearly preserves the ranking ($\rho = 0.995$). A structural proxy we had proposed (“closure”) and the in-tree `KnapsackEvaluator` do not track the peak.
3. **Use-site rematerialization for AOTAutograd** (Sections 4.1 and 5). A post-partition pass that recomputes a value again near its late use, as XLA’s rematerialization pass does [15]. It preserves the saved set exactly. On Llama it removes about half of the excess at the same budget, and its all-ops variant goes below the best stock budget; on BERT, whose excess is held dropout masks, it has little effect because it never duplicates random operations.
4. **Measurement practice** (Section 5.4). Gradient checks that caught two pitfalls of our own benchmark (a loss that is zero by construction, and dropout state), with controls that show the checks can fail.
5. **A correctness side finding** (Section 6). We independently reproduce PyTorch issue #190758: recomputed dropout under a memory budget silently corrupts gradients. We measure what its proposed fix costs: nothing measurable under Inductor, 2.3–3× peak memory on a backend without its own memory planning.

We also evaluate a liveness-aware knapsack solver as a secondary, selection-side mitigation (Section 4.2). Schedule-aware rematerialization is not a new idea: it is central to Checkmate [7], Moccasin [2], DTR [8], the algorithms of Kumar et al. [10], and XLA’s rematerialization pass [15]. Our contribution is to show where PyTorch’s production partitioner departs from it, and what bringing that idea to AOTAutograd buys and costs when measured.

Scope. All measurements use one consumer GPU (GTX 1650, 4 GB), fp32, and two small model families, without an optimizer step. We make no claims about scale, bf16, data-center GPUs or optimality (Section 8).

2 Background: how the memory budget is implemented

AOTAutograd traces the forward and backward passes into one *joint graph*. `min_cut_rematerialization_partition` splits it into a forward module and a backward module: it chooses a set of forward values to *save*, and every other forward value the backward needs is recomputed inside the backward module. The split [5, 6, 19] is a minimum cut on a flow graph whose capacities are tensor sizes scaled up with distance from the backward (and doubled for values that are not otherwise materialized), so the cut prefers saving small values close to the backward. Some nodes are *banned* from recomputation, including compute-intensive ops (matrix multiply, attention), reductions whose output is more than $4\times$ smaller than their inputs, and the random ops `native_dropout`, `rand_like` and `randn_like` (a distance-based ban also exists but cannot trigger with the default `max_dist_from_bw`).

When $0 < \text{budget} < 1$ and the more aggressive default cuts do not already fit the budget, `choose_saved_values_set` does the following (at budget 0 it simply saves the inputs):

1. Run the min-cut with aggressive options; the nodes still banned (excluding those that alias inputs or are marked must-save) are the knapsack *candidates*.
2. Solve a 0/1 knapsack over the candidates. Weight = normalized size; value = estimated recomputation runtime (FLOPs by default). Weights are normalized by the gap between the saved bytes of the minimal and the default plan, and the capacity is the budget. The capacity constrains only the candidates’ bytes, not the inputs or the other values the second min-cut saves, so the saved bytes are not guaranteed to move linearly with the budget.
3. Candidates the knapsack does not select are added to `dont_ban`, which lifts their ban.
4. Run the min-cut again with `dont_ban`. Its result is the saved set.

Two details matter for what follows. First, only banned nodes are candidates: pointwise ops such as `silu`, `mul` and `add` are never items, so the knapsack cannot see what they cost. Among the candidates, every op without a registered FLOP formula (reductions such as the RMSNorm mean or `var_mean`, dropout, embedding) is valued at 1 FLOP, so in practice only matrix multiplies and attention carry value. Second, after partitioning, `reordering_to_mimic_autograd_engine` orders the backward module. It inserts each recomputed forward subgraph immediately before its *first* backward consumer, and each forward value is recomputed at most once. The tensor then lives until its *last* consumer. (Inductor may still inline a cheap pointwise value into several consumers instead of materializing it once.) Under Inductor, the backward module is further fused, scheduled and memory-planned (including Inductor’s own `reorder_for_peak_memory`); under `aot_eager` it runs as written.

3 Diagnosis

3.1 Setup

We use two Hugging Face transformers models with random weights and synthetic token inputs of shape 2×1024 (batch 2, sequence 1024, vocabulary 1000). *Llama*: `LlamaConfig` with 32 layers, hidden size 256, MLP size 688 (SwiGLU), 8 heads, RMSNorm, SDPA attention, no dropout. *BERT*: `BertConfig` with 32 layers, hidden size 192, intermediate size 768 (GELU), 3 heads, and dropout 0.1 on attention probabilities and hidden states. All runs: NVIDIA GTX 1650 (4 GB) under WSL2, PyTorch 2.14.0+cu130, Python 3.12, fp32. A training step is one forward and one backward pass (no optimizer step). Peak memory is `torch.cuda.max_memory_allocated` during the step minus memory resident before it, which already holds the parameters, their gradients and the inputs; it is the median of 3 steps. MB denotes 10^6 bytes. Peaks are deterministic within a process; across processes the same plan’s peak varied by at most 1.5 MB (0.3%). Step time is the median of 20 iterations timed with CUDA events after warm-up. Each case re-runs stock at the end (drift at most 0.9% in reported cases); across separate sessions the same stock configuration varied by up to 0.7% on Llama and 2.7% on BERT, so step-time differences of about 1% (Llama) or 2–3% (BERT) are not meaningful.

Backends. Inductor, the default `torch.compile` backend, is our primary setting. We also report `aot_eager`, which executes the partitioner’s emitted graphs exactly as written, without fusion or rescheduling. We use it as a *partitioner-only ablation*: it removes Inductor’s decompositions, joint-graph passes, fusion and scheduling, so the partitioner sees a

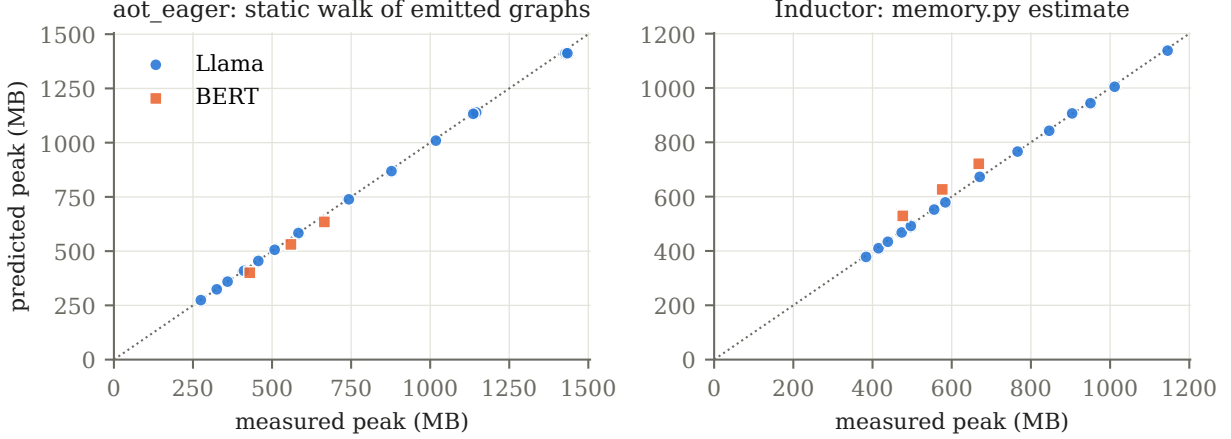


Figure 2: Predicted versus measured peak for the 14 Llama plans (natural budgets 0.05–0.35, MLP outputs forced in or out, reordering ablation) and 3 BERT plans (budgets 0.05–0.15, before the dropout fix of Section 6). Left: static walk of the emitted graphs under `aot_eager`. Right: Inductor’s own `memory.py` estimate. Dotted: $y = x$.

somewhat different joint graph (for example, 26 rather than 25 attention outputs saved at budget 0.05), and it stands in for backends without their own memory planning. It is not a deployment configuration.

3.2 The stock curve

Under Inductor the Llama peak is 904, 584, 384, 415 and 474 MB at budgets 0.05, 0.10, 0.15, 0.20 and 0.30 (`aot_eager`: 1145, 508, 275, 325, 411 MB; Fig. 1). A denser stock-only grid of 18 budgets puts the minimum at 0.13 (363 MB); the peak is 1142 MB at budget 0 and 1110 MB at budget 1. Above 0.13 the rise is expected: a larger budget saves more activations. Below 0.13 the rise is the anomaly. BERT (with the dropout fix of Section 6) has its minimum at the lowest budget of Table 2, 0.05 (475 MB), and rises below it: 609, 829 and 2071 MB at budgets 0.03, 0.02 and 0.01.

3.3 Predicting the peak of a plan

Before changing the partitioner we need to predict what a plan costs. We evaluate predictors on 14 Llama plans per backend (natural budgets 0.05–0.35, plans with MLP outputs forced into or out of the saved set, and the reordering ablation below) plus 3 BERT plans (Fig. 2).

Under Inductor, `torch/_inductor/memory.py` estimates the peak of the order its scheduler chooses. Over the 14 Llama plans its estimate has Spearman $\rho = 1.000$ with the measured peak, 0.7% median and 1.5% maximum error; on BERT it over-predicts by 8–11% but ranks the three plans correctly. A static liveness walk over the partitioner’s emitted forward and backward graphs (allocate each non-view output at its definition, free it after its last use, treat saved activations as live when the backward starts) ranks the same Llama plans almost as well ($\rho = 0.995$) but is far off in absolute terms under Inductor (19% median, 40% maximum error), because Inductor fuses and reschedules. When the backend executes the emitted graphs as written (`aot_eager`), the same walk is accurate: $\rho = 0.999$ with 0.4% median and 1.5% maximum error over the 14 Llama plans; on BERT it is low by a near-constant 29–31 MB, which preserves ranking. Our prototypes use this walk as their oracle, also under Inductor, so under Inductor they rely only on its near rank-correctness; Section 4.1 checks this for the pass’s own edits.

Two quantities one might use instead do not track the peak (`aot_eager`, 14 Llama plans). A “closure” metric (the number of unsaved candidate predecessors of each candidate, which we had earlier proposed on the issue) reaches only $\rho = 0.56$; it depends only on the saved set, so it cannot see the schedule, and it is unchanged by the reordering ablation below while the peak changes $5.2\times$. The in-tree `KnapsackEvaluator(account_for_backward_pass=True)` reaches $\rho = -0.13$. By construction it simulates only the knapsack candidates (no pointwise recomputation, no second min-cut, no reordering) and takes the knapsack’s choice as input; we did not test whether this explains its poor ranking.

Table 1: Regime sweep under Inductor (GTX 1650; 32 layers, hidden 256, batch 2, measured peaks). Saved: attention / MLP outputs in the knapsack’s choice at budget 0.02. Excess: stock peak at that budget divided by the lowest stock peak over budgets 0.02–0.30. All-ops: lowest peak of all-ops use-site rematerialization at budgets 0.05 and 0.10. The row $S = 1024$, $I = 688$ is the configuration of Table 2; this coarse grid has no budget 0.13, where the dense grid’s minimum is 363 MB (Section 3.2).

S	I	$2S/I$	saved at 0.02	best stock (budget)	excess 0.05	excess 0.02	all-ops (budget)
256	688	0.74	0 / 10	113.5 MB (0.10)	1.40×	2.07×	114.1 MB (0.05)
1024	2752	0.74	0 / 25	489.0 MB (0.05)	1.00×	2.14×	485.4 MB (0.05)
512	688	1.49	10 / 0	176.5 MB (0.15)	2.54×	2.94×	158.2 MB (0.10)
1024	1376	1.49	15 / 0	398.8 MB (0.10)	3.04×	3.88×	325.1 MB (0.05)
1024	688	2.98	10 / 0	383.7 MB (0.15)	2.36×	2.74×	292.3 MB (0.05)
1024	344	5.95	7 / 0	385.4 MB (0.20)	1.78×	2.09×	279.7 MB (0.10)
2048	688	5.95	10 / 0	913.2 MB (0.15)	1.99×	2.30×	596.7 MB (0.05)

3.4 The selection limitation

At budget 0.05 the knapsack fills the budget with attention outputs (`_scaled_dot_product_efficient_attention`: 25 under Inductor, 26 under `aot_eager`) and saves *no* MLP output. For the knapsack’s objective this is right. The partitioner’s FLOP estimate charges about $4S$ FLOPs to recompute one attention output element (causal masking is not discounted) and $2I$ for one `down_proj` output element, so with sequence length $S = 1024$ and MLP size $I = 688$ attention outputs are worth $2S/I \approx 3$ times as much per byte. (Selective recomputation in large transformers makes the opposite choice, saving linear-layer outputs and recomputing the attention core [9].) But in a pre-norm residual network the residual stream is a running sum of every layer’s `o_proj` and `down_proj` outputs. If those are not saved, recomputing the input of the first backward operation requires recomputing the residual stream through all uncut layers, including their MLP internals.

The effect is causal. Under Inductor, forcing k MLP outputs into the saved set at budget 0.05 lowers the peak monotonically: 846, 766 and 671 MB for $k = 8, 16, 24$ (stock: 904 MB). Conversely, forcing MLP outputs *out* at budget 0.15 raises the peak from 384 to 950 MB. (`aot_eager`: 1018, 878, 743 vs. 1145 MB; 275 to 1136 MB.)

This limitation depends on the regime. The ratio $2S/I$ exceeds 1 only when the sequence is long relative to the MLP size. For our models it is about 3. We tested the regime directly by varying S at $I = 688$ and I at $S = 1024$ on the same 32-layer model, under Inductor on the GPU (Table 1). The knapsack’s choice flips between ratios 0.74 and 1.49 (we tested none in between), in both sweeps: at $2S/I = 0.74$ the smallest budget saves only MLP outputs, and at every ratio above 1 only attention outputs. Above 1, budget 0.05 uses 1.8–3.0× the peak of the best budget, and all-ops use-site rematerialization (Section 4.1) goes 10–35% below that best budget. At 0.74, budget 0.05 uses 1.0–1.4× the best peak and rematerialization only matches it. The curve still rises at budget 0.02 even then (2.1×), so non-monotonicity does not need the selection limitation. A CPU run of the same sweep with the exact static oracle (`aot_eager`, every budget scored with the production knapsack solver) agrees. Stock PyTorch 2.14 neither lists the CPU attention op as compute-intensive nor gives it a FLOP formula, so on CPU attention would never be a candidate; for this run we add both, copying the CUDA kernels’ formula. There, the knapsack’s value per byte of attention outputs relative to MLP outputs tracks $2S/I$ (0.72–5.77 against 0.74–5.95), and at $2S/I = 0.74$ all-ops rematerialization brings budget 0.02 to or below the best stock budget without changing the saved set, so the rise that remains there is the lifetime limitation. For a model with $I = 14336$, as in 8B-parameter Llama-family models, the ratio passes 1 near $S \approx 7000$; we have not tested at that scale.

3.5 The lifetime limitation

Selection alone does not explain the size of the peak. In the `aot_eager` accounting at budget 0.05, tensors recomputed in the backward account for 1069 of the 1139 MB live at the peak (94%). Most of them are recomputed early, while rebuilding the residual stream for the first backward operation, and then held until their own layer’s gradient is computed, far later. Because each value is recomputed at most once and inserted at its first use, the backward module cannot recompute it again where it is needed later.

The schedule matters as much as the saved set. With the saved set fixed, disabling `reordering_to_mimic_autograd_engine` raises the budget-0.15 peak from 384 to 1012 MB under Inductor (2.6×; `aot_eager`: 275 to

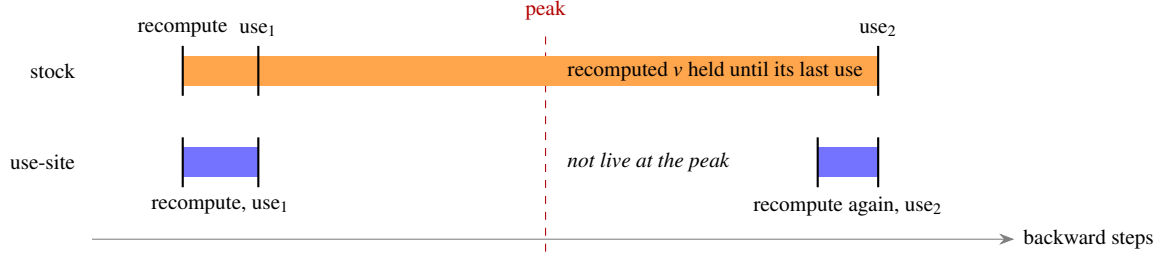


Figure 3: The lifetime limitation and the use-site pass. Stock AOTAutograd recomputes a forward value v once, before its first backward use, and holds it until its last use, across the memory peak. Use-site rematerialization recomputes v again just before the late use, so it is not live at the peak. The saved set is unchanged; the cost is the duplicated (here elementwise) work.

Algorithm 1 Use-site rematerialization of the emitted backward module (greedy).

Require: backward GraphModule G ; mode $\in \{\text{light}, \text{all}\}$; cone limit c

- 1: $T \leftarrow \text{TIMELINE}(G)$ $\triangleright$ static liveness walk: peak step p , live set at p , area
- 2: **repeat**
- 3: **for** tensors t live at p with a use after p , largest first **do**
- 4: $u \leftarrow$ first use of t after p
- 5: $C \leftarrow$ backward cone of t , stopping at saved inputs and at values still live at u for other reasons
- 6: **if** $|C| \leq c$ and every op in C is allowed by mode **then**
- 7: insert a copy of C before u ; rewire all uses of t after p to the copy
- 8: $T' \leftarrow \text{TIMELINE}(G)$
- 9: **if** $\text{peak}(T') \leq \text{peak}(T)$ and $(\text{peak}, \text{area})(T') < (\text{peak}, \text{area})(T)$ **then**
- 10: accept; remove dead code; $T \leftarrow T'$; **break**
- 11: **else**
- 12: undo
- 13: **end if**
- 14: **end if**
- 15: **end for**
- 16: **until** no change accepted, or the failure budget is exhausted

1434 MB). So the order the reordering pass produces largely determines the peak, and here it helps. By itself, though, this does not show that holding recomputed values causes the low-budget excess. The direct evidence for that is the use-site pass of Section 4.1: with the saved set unchanged, splitting those lifetimes removes about half of the excess under Inductor (Section 5).

4 Methods

4.1 Use-site rematerialization

The lifetime limitation suggests a mitigation that leaves selection alone: allow a value to be recomputed again, near its late use. This is the idea behind XLA’s `HloRematerialization` pass, which rematerializes values that are live across a point where memory exceeds a limit [15]. We implement it as a pass over AOTAutograd’s emitted backward module, run after partitioning and before the backend compiler (Fig. 3 and Algorithm 1).

The pass never changes which values the forward saves, so saved bytes and the meaning of the budget are exactly those of stock PyTorch. Its cost is duplicated computation. In *light* mode the cone may not contain compute-intensive ops (matrix multiplies, attention); it may contain any other non-mutating op: elementwise ops, reductions such as `var_mean` or `sum`, and shape and indexing ops such as `embedding` or `gather`. In *all* mode matrix multiplies may be duplicated too, trading compute for memory. Neither mode duplicates attention: every scaled-dot-product-attention op carries the `nondeterministic_seeded` tag, even without dropout, so the random-op rule below excludes it. In both

modes, ops tagged `nondeterministic_seeded`, Inductor’s random-number primitives (such as `inductor_random`), collectives, mutating ops and ops named `*_backward` are never duplicated, and cones are limited to 12 nodes. A duplicated random op is correct only if it reproduces the forward’s values, which Inductor does not currently guarantee on CUDA (Section 5.4), so the pass asserts that the number of random ops in the graph does not grow. The acceptance test uses the same static timeline as the oracle (the `fx` walk, also under Inductor), so every accepted change is predicted not to raise the peak; we then measure. Because the walk only approximates Inductor’s schedule, the predicted and measured gains differ under Inductor (Table 4). To check the pass’s decisions one by one, we compiled and measured prefixes of its edits under Inductor (the first 1/8, 1/4, 1/2, 3/4 and all of them; Llama, budgets 0.05–0.15; the pass is deterministic, so a prefix is exactly the start of the full run). At budgets 0.05 and 0.10 the walk overstates each saving $1.3\text{--}2.2\times$ (for the full light pass at budget 0.05 it predicts -515 MB against -251 MB measured), and it orders the prefixes almost correctly ($\rho = 0.94\text{--}1.00$ at budgets 0.05 and 0.10). In one case later edits undo part of the gain: the all-ops pass at 0.05 reaches -518 MB after half its edits and -480 MB after three quarters, before ending at -612 MB . At budget 0.15 it predicts -64 MB for 30 all-ops edits that change the measured peak by -0.6 MB . Inductor’s own estimator (`memory.py`) predicts the measured change of every prefix within 6 MB, so it is the better acceptance test for an Inductor version of the pass.

4.2 Liveness-aware selection (secondary)

The selection limitation can also be addressed through an existing extension point: the config option `activation_memory_budget_solver` (in `torch._functorch.config`) accepts a custom knapsack solver. Because the hook does not receive per-candidate runtimes or the number of forward outputs, our prototype also wraps two private partitioner functions to obtain them, and calls the partitioner’s private module-extraction function to emit plans. It runs the stock dynamic program, emits the plan exactly as the partitioner would (the second min-cut, module extraction and reordering), and walks it with the oracle. It then attributes the bytes of recomputed tensors live at the peak to the nearest unsaved candidates downstream of them in the recomputation graph, raises those candidates’ knapsack values, and re-solves, for up to four rounds. A bisection on the strength of the price increase then generates a few cheaper variants. Among all plans tried whose predicted peak is within 1% of the best, it returns the one that keeps the most of stock’s estimated recomputation savings, and it returns the stock plan unless the predicted gain exceeds 2%. Every returned plan satisfies the knapsack’s quantized capacity, so the budget semantics are those of stock. It used 2–7 oracle calls per graph. Choosing what to save with the schedule in view is the premise of Checkmate and Moccasin [2, 7]; we include this solver as a cheap, budget-preserving instance, not as a new idea.

5 Evaluation

Table 2 compares each method with stock *at the same budget*; Table 3 compares against the *best* stock budget, which is what a user who tunes the knob would get. Figure 4 plots the trade-off.

5.1 At the same budget

Light rematerialization. Under Inductor, where the anomaly occurs (Llama at budgets 0.05 and 0.10), the pass reduces peak memory by 27.7% and 17.3% with step-time changes of $+0.1\%$ and -0.1% , within noise. This removes about half of the excess over the lowest stock peak (363 MB at budget 0.13); light mode may not duplicate matrix multiplies, which the all-ops variant below does. Where the peak is not lifetime-driven (Llama at 0.15 and above) the pass changes nothing measurable, and light mode is never worse than stock. On BERT under Inductor it changes the peak by -0.3% , -0.2% and -3.3% at budgets 0.05, 0.10 and 0.15. An earlier version of the pass matched random ops by name and missed Inductor’s untagged seeded primitive `inductor_random`. Under Inductor it therefore regenerated dropout masks from their saved seeds a second time, next to their late uses, where stock regenerates each mask once before its first use. That version reduced BERT’s peak by 17–21% and passed the gradient check of Section 5.4, but we exclude it from all claims: whether a regenerated mask equals the forward’s depends on the dimensionality of the kernel Inductor generates for it (PyTorch issue #198333), which the pass does not control. The gap between the two versions shows where BERT’s lifetime excess under Inductor lies: in dropout masks that the backward regenerates once and holds until their last use. That excess becomes available to the pass once regeneration is reliable.

Table 2: Peak memory change versus stock at the same budget, in %, with step-time change in parentheses. Stock peak in MB. “Drift” is the step-time difference between stock and a stock re-run at the end of the case (%). Ind. = Inductor; eager = aot_eager, a partitioner-only ablation. “Light” = use-site rematerialization without matrix multiplies or attention. BERT rows are measured with the dropout fix of Section 6 emulated. The pass never duplicates random ops (Section 4.1); an earlier version that did is excluded (Section 5.1). [†]This solver plan gives incorrect gradients, caused by PyTorch issue #198333 (Section 5.4); both rows that use it are excluded from all claims.

model	backend	budget	stock	remat (light)	remat (all)	selection	selection + remat (light)	drift
Llama	Ind.	0.05	904	-27.7 (+0.1)	-67.7 (+12.7)	-29.0 (+5.6)	-44.3 (+5.7)	0.2
Llama	Ind.	0.10	584	-17.3 (-0.1)	-42.5 (+5.4)	-28.0 (+2.7)	-34.8 (+2.8)	0.1
Llama	Ind.	0.15	384	0.0 (+0.1)	-0.2 (+1.0)	-4.3 (+0.1)	-4.3 (+0.1)	0.1
Llama	Ind.	0.20	415	0.0 (+0.1)	0.0 (+0.7)	0.0 (+0.1)	0.0 (-0.1)	0.1
Llama	Ind.	0.30	474	0.0 (+0.1)	0.0 (+0.6)	0.0 (+0.1)	0.0 (-0.1)	0.1
Llama	eager	0.05	1145	-44.9 (+1.3)	-83.0 (+12.8)	-39.4 (+5.3)	-57.1 (+5.7)	0.3
Llama	eager	0.10	508	-31.3 (+0.5)	-66.4 (+4.4)	-32.3 (+1.8)	-40.1 (+1.6)	0.1
Llama	eager	0.15	275	-0.5 (+0.2)	-22.7 (+1.1)	-4.1 (-0.3)	-4.9 (-0.2)	0.2
Llama	eager	0.20	325	-0.6 (+0.3)	-16.6 (+0.9)	0.0 (+0.1)	-0.6 (+0.1)	0.0
Llama	eager	0.30	411	-0.4 (-0.1)	-5.5 (+0.5)	0.0 (-0.1)	-0.4 (-0.1)	0.3
BERT	Ind.	0.05	475	-0.3 (-0.8)	+0.4 (+4.1)	-19.7 (-2.6) [†]	-20.0 (-2.4) [†]	0.2
BERT	Ind.	0.10	575	-0.2 (-0.1)	+6.2 (+3.9)	0.0 (+0.0)	-0.2 (-0.2)	0.6
BERT	Ind.	0.15	667	-3.3 (-1.3)	+0.5 (+0.5)	0.0 (-0.7)	-3.3 (-0.6)	0.1
BERT	eager	0.05	1304	-0.2 (+0.0)	-0.2 (+0.4)	0.0 (+0.2)	-0.2 (+0.2)	0.7
BERT	eager	0.10	1432	-0.1 (+0.0)	-0.1 (-0.2)	0.0 (+0.1)	-0.1 (+0.0)	0.9
BERT	eager	0.15	1539	-0.1 (+0.0)	-0.1 (+0.0)	0.0 (+0.0)	-0.1 (+0.1)	0.2

Table 3: Under Inductor: every plan we measured whose peak is at or below the best stock plan, and not worse than another plan in both peak and step time. Δ is relative to the best stock plan among the stock rows of Table 2 and the dense stock grid (Llama: budget 0.13; BERT: 0.05; peaks measured in separate processes agree to within 1.5 MB). Step-time differences below about 1% (Llama) and 2–3% (BERT) are within run-to-run variation.

model	method	budget	peak (MB)	step (ms)	Δ peak (%)	Δ step (%)
Llama	all-ops remat	0.05	292	914	-19.7	+17.7
Llama	all-ops remat	0.10	336	826	-7.5	+6.4
Llama	stock	0.13	363	776	+0.0	+0.0
BERT	light remat	0.05	474	335	-0.3	-0.7
BERT	stock	0.05	475	337	+0.0	+0.0

All ops. Allowing matrix multiplies to be duplicated reduces the Llama peak under Inductor by 67.7% at budget 0.05, for 12.7% more step time, and makes the budget curve monotone: 292, 336, 383, 415, 474 MB across budgets 0.05–0.30 (Fig. 1). On BERT it raises the measured peak (+0.4%, +6.2% and +0.5% at budgets 0.05, 0.10 and 0.15) although the fx walk predicts a reduction (575 to 440 MB at 0.10, Table 4); under Inductor the walk can be wrong even in sign for duplicated matrix multiplies, whose scheduling it does not model.

Selection. The selection solver reduces the Llama peak under Inductor by 29.0% and 28.0% at budgets 0.05 and 0.10, for 5.6% and 2.7% more step time: unlike light rematerialization it buys memory with more expensive recomputation, saving MLP outputs and recomputing attention instead. Combined with light rematerialization it reaches −44.3% at 0.05.

Partitioner-only ablation. Under aot_eager every effect is larger, because no scheduler reorders around the emitted plan: light mode reaches −44.9%, all-ops −83.0%, and selection plus light −57.1% at Llama budget 0.05. On BERT with the dropout fix the pass has no effect there ($\leq 0.2\%$), because the peak is dominated by saved dropout masks (Section 6).

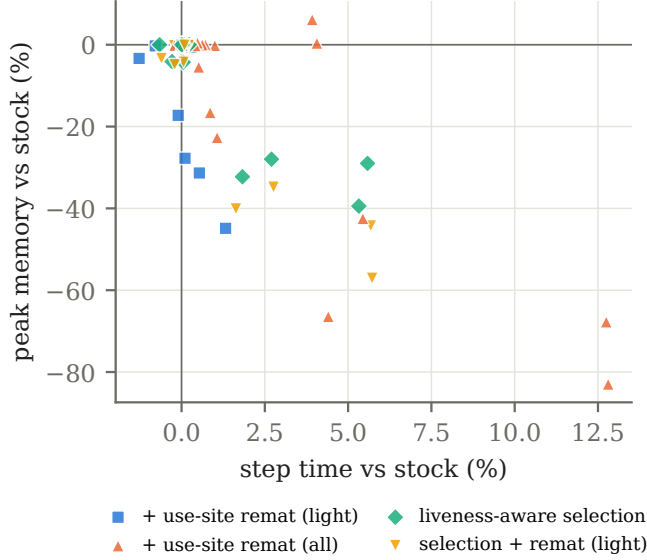


Figure 4: Peak memory versus step time, relative to stock at the same budget, for every case of Table 2 (plans with incorrect gradients omitted; both backends). Points near the origin are cases where a method had no measurable effect.

5.2 Against the best stock budget

A user who sees Fig. 1 would simply tune the budget. Table 3 asks what each method offers beyond the best stock plan, taken over the dense grid. On Llama, light rematerialization never goes below it (363 MB at budget 0.13): it removes part of the anomaly but does not beat tuning the knob. The all-ops variant does, at a price: -7.5% peak for $+6.4\%$ step time at budget 0.10, and -19.7% for $+17.7\%$ at 0.05. The selection solver’s lowest plan (367 MB at budget 0.15) is 1% above the best stock plan. On BERT no method we measured goes meaningfully below the best stock plan (light rematerialization at 0.05: -0.3%). On these models, all-ops rematerialization is the only method that extends the memory floor, at an explicit compute cost.

5.3 Compile-time overhead

Table 4 lists the work done by the pass. Our prototype re-walks the whole backward after every accepted change; in the runs of Table 2 it takes 1–48 s per graph, and the selection solver 5–72 s. Both are acceptable for an opt-in prototype and not for a default; an incremental timeline would remove most of the pass’s cost. Under Inductor the pass also enlarges the graph that Inductor compiles: Llama’s total compile time at budget 0.05 rises from 113 s (stock) to 151 s with light and 176 s with all-ops rematerialization ($+56\%$). The cost is paid once per compiled graph: the largest increase (63 s) is about 0.8% of 10,000 training steps at that configuration’s 0.81 s step, and BERT’s longest Inductor pass (27 s) about 0.8% at 0.34 s.

5.4 Correctness

A memory optimization that changes gradients is worthless, and checking for this was subtle enough that we describe it here.

- **Llama** (no randomness). Against an uncompiled fp32 eager reference at budget 0.05 under Inductor, the relative L_2 error of the full gradient is 3.33×10^{-7} for stock, 3.34×10^{-7} with light rematerialization and 3.33×10^{-7} with all-ops rematerialization (stock against a stock re-run: 1.2×10^{-9}); the differences are float reassociation from different fusion. Repeating this check at every budget from 0.05 to 0.30 on both backends gives errors of $3.30\text{--}3.38 \times 10^{-7}$ under Inductor and $1.98\text{--}2.04 \times 10^{-7}$ under `aot_eager` for stock, light and all-ops plans alike (stock against a stock re-run: at most 6.1×10^{-9}). The selection solver’s Llama plans were compared against stock, which they match to at most 2×10^{-7} .

Table 4: Work done by the use-site pass at the low budgets: predicted backward peak before and after (MB, static walk), accepted rematerializations, added work per step in units of 10^9 (a normalized proxy, not FLOPs: exact FLOPs for matrix multiplies, one unit per output element for every other op; neither mode duplicates attention, Section 4.1; step time in Table 2 is the measured cost), and pass time (s, Python, unoptimized).

model	backend	budget	mode	predicted peak	accepted	added work	time
Llama	Ind.	0.05	light	1150 $\rightarrow$ 634	180	0.54	29
Llama	Ind.	0.05	all	1150 $\rightarrow$ 200	282	149.70	46
Llama	eager	0.05	light	1139 $\rightarrow$ 628	179	0.28	30
Llama	eager	0.05	all	1139 $\rightarrow$ 193	280	149.16	48
Llama	Ind.	0.10	light	586 $\rightarrow$ 379	69	0.22	15
Llama	Ind.	0.10	all	586 $\rightarrow$ 171	115	63.55	24
Llama	eager	0.10	light	506 $\rightarrow$ 345	54	0.09	14
Llama	eager	0.10	all	506 $\rightarrow$ 167	98	51.95	22
BERT	Ind.	0.05	light	444 $\rightarrow$ 311	79	0.14	19
BERT	Ind.	0.05	all	444 $\rightarrow$ 309	109	9.49	25
BERT	Ind.	0.10	light	575 $\rightarrow$ 442	77	0.12	20
BERT	Ind.	0.10	all	575 $\rightarrow$ 440	107	9.48	27
BERT	eager	0.05	light	1271 $\rightarrow$ 1269	3	0.00	1
BERT	eager	0.05	all	1271 $\rightarrow$ 1269	3	0.00	1
BERT	eager	0.10	light	1401 $\rightarrow$ 1400	3	0.00	1
BERT	eager	0.10	all	1401 $\rightarrow$ 1400	3	0.00	1

- **BERT.** Two pitfalls nearly misled us. First, the benchmark’s training wrapper returns the sum of the final LayerNorm output, which is zero by construction; its “gradients” are mostly float residue, and comparisons of them are weak evidence. For every BERT correctness claim we therefore use a fixed random projection of the outputs as the loss. Second, dropout makes the result depend on the random number generator, so we reseed before every step and compare against budget 1.0, where nothing is recomputed. With dropout disabled, stock and light plans at budget 0.05 match budget 1.0 to 3.2×10^{-7} and 3.1×10^{-7} . With dropout enabled and the fix of Section 6 emulated, stock, light and all-ops plans (corrected pass) match budget 1.0 to $2.3\text{--}3.0 \times 10^{-7}$ at budgets 0.05, 0.10 and 0.15; the pass accepted 71–109 rematerializations in these plans, so the check is not vacuous, and the number of random ops in every backward graph was unchanged (97). The earlier version of the pass (Section 5.1), which did duplicate random ops, also passed this check ($2.6\text{--}2.8 \times 10^{-7}$); we do not take that as evidence that it is safe, because on CUDA the result depends on kernel shapes that the next item shows can change. With the fix disabled, the same test reports 5.1–5.3% errors for every plan on a smaller CPU model, so it does detect mismatched dropout masks.
- **The selection solver on BERT under Inductor.** At budget 0.05, with dropout enabled and the fix emulated, the solver’s plan gives gradients that differ from budget 1.0 by 2.0×10^{-2} , while stock differs by 2.2×10^{-7} in the same run (Table 2, †). The backward of that plan does not depend on the global random state (advancing it between forward and backward changes gradients by only 2×10^{-7}), and the *same* saved set, mapped onto the dropout-free graph (241 of 241 saved candidates mapped), gives correct gradients (2.9×10^{-7}). Reverting, one class at a time, the five operator classes in which the plan differs from stock isolates the cause: reverting only the 65 saved var_mean nodes (LayerNorm statistics) restores correct gradients (2.7×10^{-7}), while reverting any other class leaves the error at 2.0×10^{-2} . In the tested configuration, the cause is PyTorch issue #198333 [23]: since PyTorch 2.13, Inductor’s CUDA code generator draws random numbers with one function in one-dimensional kernels and another in multi-dimensional (tiled or reduction) kernels, and the two give different values for the same seed and offset. When the statistics are saved, the backward regenerates some dropout masks in one-dimensional kernels, while the forward generated all of them in multi-dimensional ones, so those masks differ. Recording which function each kernel used is consistent with this (the forward used only `tl.rand`; the backward used `rand4x` in 63 one-dimensional kernels), and disabling the one-dimensional variant (we patch Inductor’s private `Triton0overrides._can_use_4x_random` to return false; all else unchanged) makes the same plan correct: its gradients match budget 1.0 to 2.6×10^{-7} (stock: 2.5×10^{-7}), with the same 381.5 MB peak as before and no measurable change in step time. The failure is therefore not in the solver or in our emulation of the dropout fix. We still exclude the affected rows, which were measured with the faulty path enabled.

Negative results. A variant of the selection solver that additionally forced fractions of repeated-structure candidate classes, ranked by peak attribution, gave identical measured peaks in all six cases we tried, at $1.4\text{--}2.2\times$ the solver time. An earlier, broader class-forcing search found gains the repricing solver misses (Llama aot_eager: -10.2% , -15.8% and -5.4% at budgets 0.15, 0.20 and 0.30), so the solver is not close to optimal there. Finally, the default FLOP-based runtime estimate is a poor proxy for cost: for one Llama plan it predicted keeping only 40% of stock’s recomputation savings, yet the measured step time rose by only 5%; PyTorch’s profiling-based estimator predicted that cost within about 5%.

6 Side finding: randomness under memory budgets

While validating BERT we found that whenever the partitioner reaches its knapsack path (or budget 0), the backward module can recompute dropout with fresh randomness, so its masks differ from the forward’s and gradients are silently wrong. This is the open PyTorch issue #190758 [21], which already identifies the Inductor path through `prims.inductor_seeds` and proposes saving the outputs of random ops [13]; replaying the random state is an alternative [28]. We reproduced it independently and measured three things.

1. **Reproducers.** A stack of eight Linear–GELU–Dropout blocks on CPU has a 21.5% gradient error under Inductor at budget 0, confirmed by a finite-difference check; under aot_eager gradients are 18–22% wrong at every budget below 1 that we tested (0.9, 0.5, 0.1, 0). On a 4-layer BERT on CPU under Inductor, advancing the global random state between forward and backward changes the budget-0.05 gradient by 3.7% (budget 1: 2.7×10^{-7}). On the GPU BERT, the budget-0.05 gradient differs from budget 1.0 by 2.0×10^{-2} with identical forward masks.
2. **The proposed fix works.** Emulating it (no op tagged `nondeterministic_seeded` is recomputed) reduces the CPU BERT error from 3.5% to at most 4×10^{-7} .
3. **What the fix costs.** Under Inductor it saves only the small seeds tensor: at budgets 0.05, 0.10 and 0.15 the stock BERT peak changes by less than the run-to-run variation across processes (at most 1.5 MB). On a backend that executes the emitted graph as written (aot_eager) the fix saves full dropout masks, and the peak rises from 430 to 1304 MB at budget 0.05 (560 to 1432 and 665 to 1539 MB at 0.10 and 0.15). This is quantitative input for choosing between saving random outputs and replaying random state.

All BERT numbers in Table 2 are measured with the fix emulated, so they describe the corrected baseline.

Memory budgets also expose a second, independent random-number bug. The wrong solver gradients of Section 5.4 come from PyTorch issue #198333 [23], in which Inductor’s CUDA code regenerates dropout masks with a different random function when the backward kernel’s dimensionality differs from the forward’s. The issue reports it through `torch.utils.checkpoint`; we reach it without any checkpoint API, through the memory-budget partitioner with the proposed fix for #190758 applied, by saving a different set of activations.

7 Related work

Schedule-aware rematerialization. Checkmate formulates rematerialization jointly with the schedule as a mixed-integer program and finds optimal plans offline [7]; Moccasin reformulates it with retention intervals and constraint programming [2]. MONeT jointly optimizes checkpointing and operator implementations [25], POET adds paging for edge devices [17], and Rockmate combines block dynamic programming with integer programming for PyTorch models [32]. These methods are optimal or near-optimal but rely on offline solvers; our setting is a production compiler with seconds of compile-time budget and an existing API contract. XLA’s rematerialization pass [15] is the direct ancestor of our use-site pass, and recomputing a value near each use instead of extending its live range is classic register-allocation rematerialization [3]; Kumar et al. [10] give rematerialization schedules with provable guarantees for graphs of bounded treewidth. Echo [33] and SuperNeurons [30] recompute only cheap layers in the backward, as our light mode does. He and Yu [6] make AOTAutograd’s min-cut fusion-aware, the partitioner we study.

Dynamic and allocator-aware approaches. DTR evicts and recomputes at runtime [8]; Coop makes this fragmentation-aware [31]. A recent simulator study of DTR reports regime switching and a “feasibility inversion” in which training is possible at one budget, impossible at a larger one and possible again at a larger one still [16]. That non-monotonicity has a different mechanism, in a different system, from ours.

Ordering and layout. OLLA [29] and ROAM [26] reduce peak memory by optimizing operator order and tensor placement without recomputation; Inductor’s `reorder_for_peak_memory` is a production instance. Inductor’s reordering is enabled in all our Inductor runs and does not undo the anomaly.

PyTorch. The memory-budget API is described in [24] and its knapsack is implemented in [19]; Maczan [12] reduce the knapsack solver’s own memory, and issue #149258 [20] reports the same residual-stream recomputation chain from the speed side. Several tools choose what to save against an absolute per-device memory budget: PyTorch’s `sac_milp` formulation for selective activation checkpointing [18], and a budget-aware keep/recompute/offload policy proposed for torchtitan [14]. Such tools decide *what* to save; the placement of recomputation inside the backward is still left to the partitioner, as it is for PyTorch’s tag-based rematerialization pass (`remat_using_tags_for_fwd_loss_bwd_graph`), which likewise recomputes each node once at its first backward use. The use-site pass is complementary to all of them.

Concurrent work. We reimplemented the greedy save-set selector of Somers [27] from its published description and ran it on a CPU toy model (16-layer Llama-style model, `aot_eager`, exact static oracle): at matched peak it recomputes $1.3\text{--}17\times$ fewer FLOPs than our liveness-aware solver, and use-site rematerialization lowers its final plan slightly further (58.9→58.3 MB), consistent with selection and lifetime being separate limitations. Our reimplementation may differ from the original, and we have not run it on our GPU setup.

8 Limitations and future work

- **Scale.** One 4 GB consumer GPU, fp32, two small model families, no optimizer step. Whether the anomaly and the gains persist at 1–8B parameters, in bf16 and on data-center GPUs is untested. The selection limitation appears only when $2S/I > 1$ (Table 1); that sweep varies S and I on the 32-layer, hidden-256 model only.
- **End-to-end metric.** We report peak memory and step time, not the largest batch or sequence that fits.
- **Baselines.** We compare against stock at the same and at the best budget, but not against optimal plans (Checkmate or Moccasin on small graphs), manual checkpointing policies, or absolute-budget planners.
- **Oracle under Inductor.** Our prototypes use the fx walk as their oracle even under Inductor, where it only nearly preserves the ranking of plans and overstates the pass’s savings $1.3\text{--}2.2\times$ at budgets 0.05 and 0.10 (Section 4.1). Inductor’s own estimator, which predicts the effect of the pass’s edits within 6 MB, would be the better choice.
- **Safety of the pass beyond our graphs.** Duplication is safe in the functional backward graphs we tested; graphs with in-place mutation of storage (for example under fully sharded data parallelism) would need storage-aware checks, and the pass’s list of compute-intensive ops is matched by name.
- **Compile time.** The prototypes add tens of seconds per graph (up to +56% total compile time for Llama under Inductor).
- **Random-number regeneration.** The pass never duplicates random operations, which forgoes BERT’s lifetime gain (Section 5.1). Under a budget, Inductor already regenerates each dropout mask once from its saved seed; regenerating it again near a late use relies on the same mechanism, but on CUDA it is reliable only once PyTorch issue #198333 is fixed. We did not test the earlier version of the pass with the affected code path disabled.
- **PyTorch issue #198333.** Our results use PyTorch 2.14, which has this bug. Until it is fixed, plans that change the shape of the kernels that regenerate dropout masks can give wrong gradients on CUDA; this is what happened to the solver on BERT (Section 5.4). Disabling the affected code path, as we did by patching a private Inductor function, is a workaround; we did not test the `align_random_eager` setting that the issue suggests.

The natural next steps are an incremental version of the pass that uses Inductor’s estimator, combining it with a selection policy evaluated after rematerialization, evaluation on larger models, and a discussion with PyTorch maintainers about offering the pass behind a flag.

Reproducibility

Scripts, per-run JSON results, logs and this paper’s source are available at <https://github.com/sidddd17/less-budget-more-memory>. All tables and figures are generated by a script from the per-run JSON files, and every other number is in a JSON or log file produced by the script named in the repository’s README. The GPU results require a CUDA GPU to rerun; the CPU results run on any machine.

Acknowledgments

We thank Aiswarya for lending the laptop with the GTX 1650 GPU on which all experiments were run.

References

- [1] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, et al. PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS)*, 2024. doi: 10.1145/3620665.3640366.
- [2] Burak Bartan, Haoming Li, Harris Teague, Christopher Lott, and Bistra Dilkina. Moccasin: Efficient tensor rematerialization for neural networks. In *International Conference on Machine Learning (ICML)*, 2023.
- [3] Preston Briggs, Keith D. Cooper, and Linda Torczon. Rematerialization. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 1992.
- [4] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016.
- [5] Horace He. Min-cut optimal(*) recomputation (i.e. activation checkpointing) with AOTAutograd. PyTorch dev-discuss forum, <https://dev-discuss.pytorch.org/t/min-cut-optimal-recomputation-i-e-a-ctivation-checkpointing-with-aotautograd/467>, 2022.
- [6] Horace He and Shangdi Yu. Transcending runtime-memory tradeoffs in checkpointing by being fusion aware. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 5, pages 414–427, 2023.
- [7] Paras Jain, Ajay Jain, Aniruddha Nrusimha, Amir Gholami, Pieter Abbeel, Kurt Keutzer, Ion Stoica, and Joseph E. Gonzalez. Checkmate: Breaking the memory wall with optimal tensor rematerialization. In *Proceedings of Machine Learning and Systems (MLSys)*, 2020.
- [8] Marisa Kirisame, Steven Lyubomirsky, Altan Haan, Jennifer Brennan, Mike He, Jared Roesch, Tianqi Chen, and Zachary Tatlock. Dynamic tensor rematerialization. In *International Conference on Learning Representations (ICLR)*, 2021.
- [9] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. Reducing activation recomputation in large transformer models. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 5, 2023.
- [10] Ravi Kumar, Manish Purohit, Zoya Svitkina, Erik Vee, and Joshua R. Wang. Efficient rematerialization for deep networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [11] lonewolf15116. activation_memory_budget sweep: GPT-style model, RTX 5050 laptop (pytorch#197838). GitHub gist, linked from PyTorch issue #197838, <https://gist.github.com/lonewolf15116/1af2cd9c8d56c076755590f9d6eb06a6>, 2026.
- [12] Jędrzej Maczan. Memory-efficient activation checkpointing with sliding window and Hirschberg’s algorithm for 0/1 knapsack solving in PyTorch. *arXiv preprint arXiv:2608.08740*, 2026.

- [13] Hussain Nizamani. Save forward RNG outputs when the memory-budget partitioner forces recompute (PyTorch draft pull request #190759). <https://github.com/pytorch/pytorch/pull/190759>, 2026.
- [14] nnurlan008. graph_trainer: budget-aware activation memory policy (auto_perf_maxing) (torch titan pull request #4636, open). <https://github.com/pytorch/torch titan/pull/4636>, 2026.
- [15] OpenXLA contributors. HloRematerialization pass (hlo_rematerization.h). XLA compiler source, https://github.com/openxla/xla/blob/main/xla/hlo/transforms/simplifiers/hlo_rematerization.h, 2026.
- [16] Mahesh Reddy Pagadala. Deterministic regime switching and feasibility inversion in dynamic tensor rematerialization. *arXiv preprint arXiv:2609.31250*, 2026.
- [17] Shishir G. Patil, Paras Jain, Prabal Dutta, Ion Stoica, and Joseph E. Gonzalez. POET: Training neural networks on tiny devices with integrated rematerialization and paging. In *International Conference on Machine Learning (ICML)*, 2022.
- [18] PyTorch contributors. sac_milp: a mixed-integer program for selective activation checkpointing (torch.distributed._tools.sac_ilp). PyTorch 2.14 source, https://github.com/pytorch/pytorch/blob/v2.14.0/torch/distributed/_tools/sac_ilp.py, 2026.
- [19] PyTorch contributors. torch/_functorch/partitioners.py. PyTorch 2.14.0 source, https://github.com/pytorch/pytorch/blob/v2.14.0/torch/_functorch/partitioners.py, 2026.
- [20] PyTorch issue #149258. Auto-selective activation checkpointing is not optimal for speed (issue with min_cut_rematerization_partition). <https://github.com/pytorch/pytorch/issues/149258>, 2025.
- [21] PyTorch issue #190758. Memory-budget partitioner recomputes RNG ops with fresh randomness in backward, silently corrupting gradients at any activation_memory_budget < 1.0. <https://github.com/pytorch/pytorch/issues/190758>, 2026.
- [22] PyTorch issue #197838. activation_memory_budget: physical peak allocated memory is non-monotonic across budgets. <https://github.com/pytorch/pytorch/issues/197838>, 2026.
- [23] PyTorch issue #198333. Since 2.13, inductor’s 4x random path corrupts CUDA dropout masks. <https://github.com/pytorch/pytorch/issues/198333>, 2026.
- [24] PyTorch Team. Current and new activation checkpointing techniques in PyTorch. <https://pytorch.org/blog/activation-checkpointing-techniques/>, 2025.
- [25] Aashaka Shah, Chao-Yuan Wu, Jayashree Mohan, Vijay Chidambaram, and Philipp Krähenbühl. Memory optimization for deep networks. In *International Conference on Learning Representations (ICLR)*, 2021.
- [26] Huiyao Shu, Ang Wang, Ziji Shi, Hanyu Zhao, Yong Li, and Lu Lu. ROAM: Memory-efficient large DNN training via optimized operator ordering and memory layout. *arXiv preprint arXiv:2310.19295*, 2023.
- [27] James Somers. Trading off compute for memory with activation checkpointing. Jane Street Tech Blog, <https://blog.janestreet.com/trading-off-compute-for-memory-with-activation-checkpointing/>, 2026. 30 September 2026.
- [28] soulitzer. [AOTAutograd] replay RNG state for memory-budget rematerialization (PyTorch draft pull request #191684). <https://github.com/pytorch/pytorch/pull/191684>, 2026.
- [29] Benoit Steiner, Mostafa Elhoushi, Jacob Kahn, and James Hegarty. OLLA: Optimizing the lifetime and location of arrays to reduce the memory usage of neural networks. *arXiv preprint arXiv:2210.12924*, 2022.
- [30] Linnan Wang, Jinmian Ye, Yiyang Zhao, Wei Wu, Ang Li, Shuaiwen Leon Song, Zenglin Xu, and Tim Kraska. SuperNeurons: Dynamic GPU memory management for training deep neural networks. In *Proceedings of the 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, 2018. doi: 10.1145/3178487.3178491.

- [31] Jianhao Zhang, Shihan Ma, Peihong Liu, and Jinhui Yuan. Coop: Memory is not a commodity. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [32] Xunyi Zhao, Théotime Le Hellard, Lionel Eyraud-Dubois, Julia Gusak, and Olivier Beaumont. Rockmate: an efficient, fast, automatic and generic tool for re-materialization in PyTorch. In *International Conference on Machine Learning (ICML)*, 2023.
- [33] Bojian Zheng, Nandita Vijaykumar, and Gennady Pekhimenko. Echo: Compiler-based GPU memory footprint reduction for LSTM RNN training. In *ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020.